

AERIS CHECKPOINT 5.5 — Pre-Aero Hardening Completed, Ready for Aero Architecture

Use this as the starting prompt for the next chat.

CONTEXT — WHAT AERIS IS

We are building AERIS, a modular aircraft-design software platform.

It is not a bag of scripts.

It is a structured engineering pipeline intended for:

- deterministic geometry generation
- modular sampling
- structured dataset generation
- future modular aerodynamic analysis
- future ML-ready data products
- future certification-aware traceable workflows

The architecture is intentionally layered:

- framework layer
- implementation layer

- pipeline/orchestration layer

- CLI layer

The system is meant to behave like a modular factory:

interfaces define machine types, implementations plug into registries, pipelines orchestrate production lines, and the CLI acts as the control panel. This software philosophy is already established in the project architecture explanation and checkpoint reports.

CURRENT STATE — WHAT IS NOW TRUE

1. CLI backbone exists and works

The CLI command groups currently include:

- version

- geometry

- dataset

- pipeline

2. Geometry generator architecture is real

A formal geometry generator abstraction exists.

Generator resolution is registry-based.

The active generator is:

- bwb_segmented_v1

The geometry generator contract includes:

- build_config(raw_config)
- sample_one(config, seed)
- run_full_case(sample, config, output_dir)
- summarize_case(case)

3. Sampling architecture is modular and validated

A dataset sampler abstraction and registry exist.

Validated samplers:

- lhs_v1
- random_v1

Important cleanup was already done:

- legacy lhs_seed path was removed from the production spine
- real sampler control now uses sampler_seed
- sampler choice is independent of geometry generator choice

4. Deterministic geometry behavior is enforced

The current design rule is:

- RNG is used only to create explicit design samples
- once a design sample exists, geometry realization is deterministic from that sample

This is essential for reproducibility, ML dataset traceability, and future certification-style workflows.

5. Dataset pipeline is the real production path

The current working dataset generation path is:

raw config

- resolve generator
- build typed config
- resolve sampler
- generate N explicit design samples
- generator.run_full_case() for each sample
- write geometry artifacts
- append metadata rows
- append failure rows if needed

→ write dataset manifest and logs

This means dataset generation does NOT reimplement geometry logic.

The generator remains the single deterministic geometry realization authority.

6. Single-geometry path is also working

The current single-case production path is:

aeris geometry generate --config ...

This:

- loads YAML
- resolves generator
- builds typed config
- creates one explicit design sample
- generates one geometry case
- writes run folder artifacts and manifest

7. File outputs are structured

Single-case output goes under:

data/runs/<timestamp>_geometry_<config>/

Dataset output goes under:

data/datasets/<dataset_name>/

Dataset contents include:

- geometry/
- metadata.csv
- failures.csv
- manifest
- copied input config
- resolved config snapshot
- logs

8. Pre-aero hardening checks are complete

The following were validated successfully:

- bad-config failure handling
- missing-config CLI rejection
- invalid sampler failure handling
- modular sampler swap (lhs_v1 ↔ random_v1)

- 500-case no-plot stress-tested geometry dataset generation
- AeroSandbox-enabled batch generation validation

9. Stress-test results

The geometry-only no-plot stress path is stable:

- 500 requested
- 500 attempted
- 500 succeeded
- 0 failed
- no missing files
- no duplicate IDs
- metadata row count correct
- failure row count correct

Plotting was identified as major avoidable overhead.

No-plot mode is now understood as the correct production hot path for large batch generation.

10. AeroSandbox geometry build path is validated

A smaller AeroSandbox-enabled dataset run confirmed:

- aspect_ratio_aerosandbox populated
- n_xsecs_aerosandbox populated
- no missing files
- clean metadata integrity

This means the system already supports:

- pure geometry/export datasets
- geometry + AeroSandbox-object construction datasets

CURRENT LIMITATIONS / HONEST GAPS

1. Only one true geometry family exists right now

Current active family:

- bwb_segmented_v1

2. Sampler plugins are architecturally modular, but current implementations are still practically tied to the BWB config/sample types

3. Export / geometry-view adapters are not yet formal plugin families

AeroSandbox geometry construction exists, but there is not yet a formal geometry-view/export adapter seam.

4. Dataset builders are not yet formal plugin families

Current dataset generation writes geometry metadata artifacts.

Future builders for SDF, graph, aero-label, or richer representations are not yet formalized inside AERIS.

5. Aero architecture is still only a shell

The repo has an aero folder scaffold, but the actual operational modular aero workflow is not implemented yet.

6. One known geometry implementation issue exists

In high dihedral-gradient cases, the mirrored left/right wing centerline can show a seam/gap near the root due to symmetric loft closure behavior.

Likely fixes later:

- root dihedral blending
- or explicit full-wing construction rather than relying on symmetric mirror closure

LEGACY SCRIPTS — WHAT THEY TAUGHT US

Older prototype/reference scripts proved why modularity is necessary.

They revealed future needs for:

- alternate geometry families (e.g. CST-based)
- geometry extraction policies
- multiple geometry views/export adapters (AeroSandbox / OpenVSP / RCAIDE)
- AVL-based aero solver adapters with raw outputs and strip data

- feature builders
- SDF / graph / mesh dataset builders
- downstream ML consumers

These scripts should be treated as migration references, not copied directly into AERIS.

They justify future plugin families and extension seams, but they should not drag the project back into script-spaghetti architecture.

WHAT SHOULD BE PREPARED NOW VS LATER

Prepare NOW:

1. Aero solver plugin architecture
2. Lightweight geometry-view/export seam
3. Keep future dataset-builder modularity in the design

Do NOT fully implement yet:

1. Extraction-policy registry
2. Reverse-fitting / inverse geometry plugins
3. ML model/training plugin systems

NEXT PHASE — CHECKPOINT 6

Now design the real aero architecture.

The next chat must produce a clean modular aero system consistent with existing AERIS philosophy.

Target immediate deliverables:

1. aero/base.py
2. aero/models.py
3. aero/registry.py
4. aero/validation.py
5. aero/solvers/aerosandbox_avl.py

The first aero system must define:

- aero input representation
- flight condition model
- standardized aero result model
- failure model
- solver registry pattern
- first solver implementation

Initial aero v1 should focus on:

- CL
- CD
- Cm
- L/D
- solver status
- provenance/runtime metadata

Critical design decisions for the next chat:

1. What is the canonical geometry input to the aero layer?
2. What are the standardized minimum outputs?
3. How are failures represented?
4. How are solver-specific details kept behind the common interface?
5. How should future geometry-view/export seams connect to the aero layer?

IMPORTANT INSTRUCTION FOR THE NEXT CHAT

Continue from this exact validated state.

Do not restart from generic advice.

Do not collapse into script-style direct solver calls.

Do not overdesign speculative plugin families.

Do not break the current geometry/dataset architecture.

Act as a system architect extending a working modular platform into its first real aero phase.

The current foundation is stable.

The next job is to add physics without destroying that stability.